大模型RL对齐算法复习手册

大模型强化学习对齐算法 · 复习手册

> 覆盖：PPO · DPO · GRPO · DAPO · Dr.GRPO · GSPO · RLOO · RLVR

目录

- 第零章：全局视野
- 第一章：PPO — 奠基者
- 第二章：DPO — 极简替代者
- 第三章：GRPO — 省显存秘籍
- 第四章：DAPO — 长推理专家
- 第五章：Dr.GRPO — 纠偏专家
- 第六章：GSPO — 序列级颠覆者
- 第七章：面试速查

第零章：全局视野

三阶段训练流水线

阶段	名称	目标	核心技术
Stage 1	预训练	学习语言知识与世界常识	自回归语言模型
Stage 2	SFT 微调	遵循指令、对话格式	Teacher Forcing + 交叉熵
Stage 3	**对齐 / RLHF**	**有用、无害、符合价值观**	**PPO / DPO / GRPO...**

> ⚠ **面试惯例**：面试官默认你懂 Stage 1-2，考点全在 Stage 3。

RL 核心术语

术语	符号	在大模型中对应什么
状态	s	Prompt + 已生成的 token 上下文
动作	a	预测并输出下一个 token
策略	π_θ	大模型本身（参数 θ ）
奖励	R	Reward Model 对整条回答的打分
价值	$V(s)$	Critic 对「从当前状态出发未来期望总奖励」的预测
优势	A_t	= 实际回报 − 基准分；这步比平均好多少

重要性采样

在 GRPO/PPO 这类强化学习算法里，训练时有个核心问题：

> **用旧策略（old policy）采样的数据，来更新新策略（new policy）**

因为新旧策略不同，需要做"重要性权重"修正：

$$w = \frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}$$

为什么只用 SFT 不够?

SFT 本质是**行为克隆**，三个根本缺陷：

1. **分布漂移**：推理时生成了训练集外的 token，模型就「迷路」，没有纠错机制。
2. **无法优化复杂目标**：「有多有用」「有多安全」这类问题难以转化成监督标注。
3. **天花板受限于数据质量**：SFT 只能学到训练集上限，RL 的 trial-and-error 可以超越它。

第一章：PPO

PPO（Proximal Policy Optimization，**近端策略优化**）是大模型 RLHF 的基石，理解它是理解所有后续算法的前提。

1.1 四模型架构

模型	是否训练	作用
🎭 **Actor**（演员）	✅ 训练	生成回答，最终部署的就是它
⚖️ **Critic**（评论家）	✅ 训练	预测 $V(s)$，提供 GAE 所需基准分
🏆 **Reward Model**	❌ 冻结	对完整回答打最终分
⚓ **Reference Model**	❌ 冻结	SFT 后模型的克隆，通过 KL 散度防止走偏

7B 模型全参数 PPO 显存估算：

组成	估算
Actor（参数 + 梯度 + Adam 状态）	≈ 84 GB
Critic（可共享主干）	≈ 40 GB
Reference + RM（冻结，只前向）	≈ 28 GB
合计	**≈ 150 GB，约需 2-3 张 A100 80G**

> 省显存方案：LoRA（梯度+Adam 状态从 70G 降到 <5G）、DeepSpeed ZeRO-3（参数分片）、梯度检查点。

1.2 重要性采样与概率比值

朴素策略梯度每次更新后必须重新采样，成本极高。PPO 引入**重要性采样**复用旧数据：

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

$r_t > 1$：新策略比旧策略更倾向生成这个 token

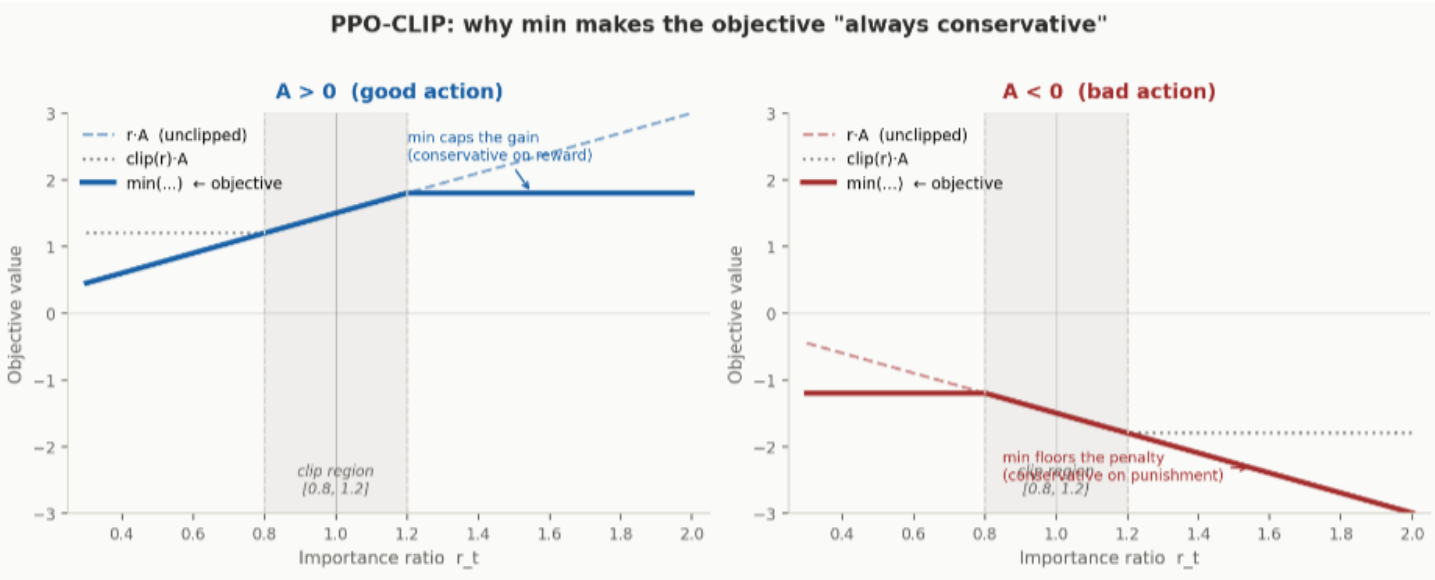
$r_t < 1$：新策略比旧策略更不倾向生成这个 token

$r_t = 1$：两个策略对这个 token 的看法一致

1.3 PPO-CLIP 目标函数

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min(r_t(\theta) \cdot A_t,\ \text{clip}(r_t(\theta),\ 1-\varepsilon,\ 1+\varepsilon) \cdot A_t)\right]$$

其中 $\varepsilon = 0.2$，即截断范围 $[0.8,\ 1.2]$。



> 🚨 **面试陷阱：为什么取 `min` 而不是直接 `clip`？**
>
> `min` 让目标函数「总是悲观」：

- 好动作（$A > 0$）：`min` 选更小值，防止过度强化
- 坏动作（$A < 0$）：`min` 选绝对值更小的值，防止过度惩罚

> 直接 `clip` 只截断比值，无法同时限制两个方向的梯度更新。

1.4 GAE 广义优势估计

核心问题：用 Monte Carlo 方差大，用 TD 偏差大，GAE 用 λ 在两者间插值：

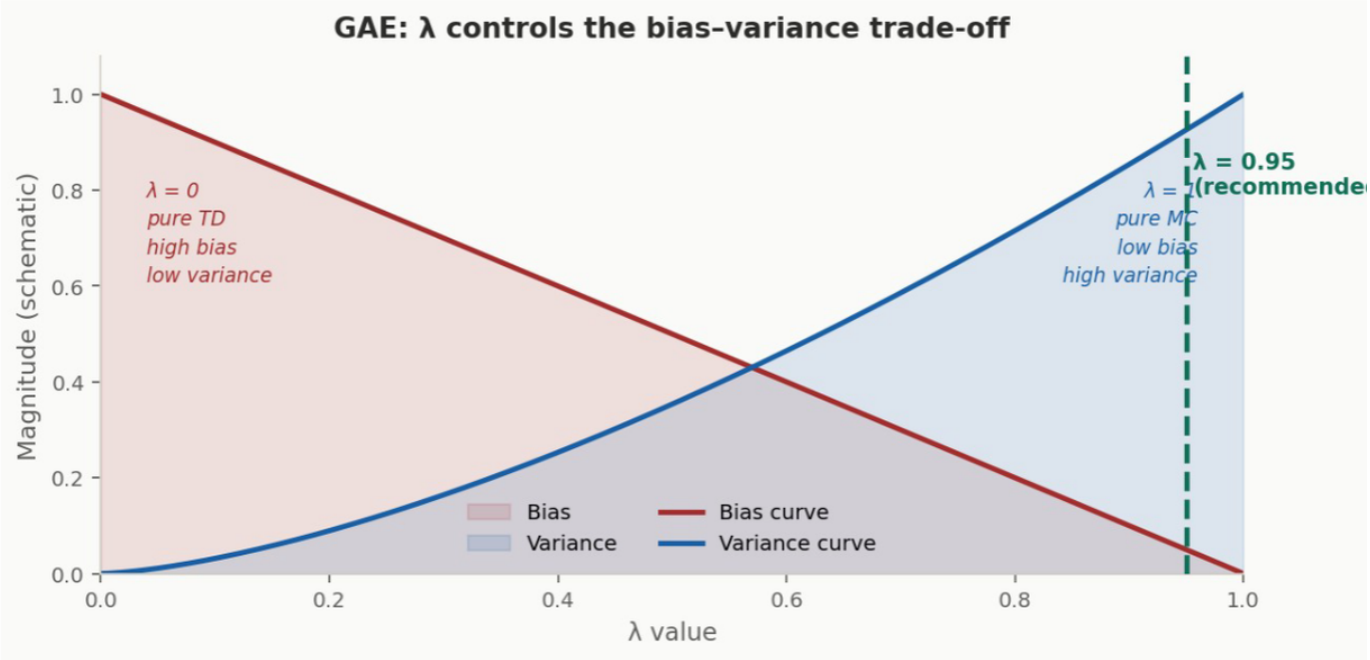
TD 残差（每步即时误差）：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

直觉：预期得 $V(s_t)$，实际得 $r_t + \gamma V(s_{t+1})$，差值就是这步的「超/低预期」。

GAE 优势（指数衰减加权未来 TD 残差）：

$$A_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \cdot \delta_{t+l} = \delta_t + \gamma\lambda \cdot \delta_{t+1} + (\gamma\lambda)^2 \cdot \delta_{t+2} + \cdots$$



λ 取值	等效于	优点	缺点
$\lambda = 0$	纯 TD（只看 1 步）	方差极小，训练稳定	完全依赖 Critic 精度，偏差大
$\lambda = 1$	纯 MC（看到结尾）	偏差极小，估计准确	方差大，训练抖动
$\lambda = 0.95$	**推荐默认，约看 20 步**	方差/偏差平衡最优	超参数需调

逆序递推实现（代码高效写法）：

代码块

```
gae = 0
for t in reversed(range(T)):
    delta = rewards[t] + gamma * values[t+1] - values[t]
    gae   = delta + gamma * lam * gae    # 滚动累积
    advantages[t] = gae
```

1.5 KL 散度惩罚与 Reward Hacking

带 KL 惩罚的总奖励：

$$R_{\text{total}} = R_{\text{RM}} - \beta \cdot D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$$

$$D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) = \sum_a \pi_\theta(a|s) \log \frac{\pi_\theta(a|s)}{\pi_{\text{ref}}(a|s)}$$

$D_{\mathrm{KL}} = 0$：Actor 与 Reference 完全一致（没学到任何新东西）

D_{KL} 越大：偏离初始模型越远（可能在作弊）

为什么需要 KL 惩罚？ 没有它，Actor 会发现「礼貌废话得高分」，对简单问题输出几千字废话——这叫 **Reward Hacking（奖励作弊）**。

β 的影响：太大则策略被锁死，太小则走火入魔。典型取值 0.01～0.1。

第二章：DPO

DPO（Direct Preference Optimization，**直接偏好优化**）出发点：PPO 太复杂，能不能不用强化学习也能对齐？

2.1 核心思想

DPO 证明了一个数学等价性：**语言模型自身的输出概率差，就等价于 Reward Model 的打分。** 只需要 `(chosen, rejected)` 对比数据，不需要 RM，也不需要 Critic。

模型	状态
Actor π_θ	✅ 训练
Reference π_{ref}	✅ 冻结
~~Reward Model~~	❌ 已砍掉
~~Critic~~	❌ 已砍掉

2.2 DPO 损失函数

推导逻辑：

好回答的相对偏好增量（相比初始模型，当前模型对好回答「喜欢程度的变化」）：

$$\Delta_w = \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)}$$

坏回答的相对偏好增量：

$$\Delta_l = \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}$$

DPO 损失（最大化两者差距）：

$$\mathcal{L}_{\mathrm{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma(\beta \cdot \Delta_w - \beta \cdot \Delta_l)\right]$$

> 💡 **隐式 KL（面试加分点）**：β 参数在数学推导中等价于 KL 惩罚系数，已内嵌在损失函数里，不需要显式写 KL 项。DPO 用 2 个模型照样能防止走偏。

2.3 如何计算某条回答的概率？

Teacher Forcing + 对数似然（逐 token 累加）：

$$\log P(y \mid x) = \sum_{i=1}^{L} \log P(\text{token}_i \mid x,\ \text{token}_{1..i-1})$$

为什么用 log 加法而不是连乘？几千个概率连乘会产生数值下溢（浮点无法表示），对数把乘法变加法完美解决。

2.4 PPO vs DPO

维度	PPO	DPO
训练范式	在线 RL（模型自采样）	离线监督学习（固定数据集）
模型数量	4 个	2 个
显存压力	极高（~150 GB / 7B）	极低（约等于 SFT）
训练稳定性	复杂，需调 β 、GAE 等	极稳定，基本不崩溃
数据需求	只需 Prompt，模型自采样	需要 (chosen, rejected) 对
泛化能力	强（在线探索）	弱（容易过拟合离线数据）
适用场景	推理/数学/代码（可自动验证）	写作/对话/偏好对齐

> ✅ **必考题：为什么 DeepSeek 训推理模型不用 DPO？**
>
> 数学题标注成本极高：人类专家要逐行检查找错误，还要故意写一个「看似正确但有细微错误」的 rejected 样本。而 GRPO + RLVR 只需一个代码验证函数（`answer == ground_truth`）自动打分，成本低两个数量级。

2.5 DPO 的数据集长什么样

`prompt` (或 `context`): 用户的输入指令或多轮对话的历史上下文。

`chosen`: 被人类标注员或更强的 AI（如 GPT-4）偏好的回复（即“好”的回答）。

`rejected`: 不被偏好的回复（即“差”的回答，可能包含幻觉、逻辑错误、不够礼貌或完全答非所问）。

典型的数据示例（JSON格式）

JSON

```
{
  "prompt": "请给我写一首关于春天的七言绝句。",
  "chosen": "草长莺飞二月天，拂堤杨柳醉春烟。儿童散学归来早，忙趁东风放纸鸢。",
  "rejected": "春天是一个美丽的季节，花儿都开了，树叶也绿了。我很喜欢春天。"
}
```

注意： 为了让模型学到有用的特征，`chosen` 和 `rejected` 的差异不能仅仅是“字数长短”，而应该是“质量高低”。高质量的 DPO 数据集往往会让 `chosen` 和 `rejected` 在长度上保持相近，以防止模型产生“越长越好”的错误偏好（Length Bias）。

2.6 DPO的改进方法

DPO 的主流改进方法主要围绕**省资源、降门槛、提上限**三个方向，简述如下：

- **省显存（无需参考模型 Reference-Free）：**
  - **ORPO：** 把 SFT（监督微调）和偏好对齐合二为一，直接砍掉庞大的参考模型，大幅节约显存。
  - **SimPO：** 同样无需参考模型，且引入了“长度归一化”，专治模型喜欢“废话连篇”（偏好长文本）的顽疾。
- **降门槛（不依赖成对数据）：**
  - **KTO：** 不需要严格的 `（获胜， 失败）` 配对数据。只要有零散的单条回答加上“点赞/点踩”记录就能训练，极其适合现成的用户反馈数据。
- **提上限（在线/迭代自我进化）：**
  - **Online / Iterative DPO：** 抛弃静态离线数据。让模型自己实时生成回答，由外部裁判（强 LLM 或奖励模型）动态打分并构造新数据。这是目前 Llama 3 等顶尖模型冲榜的核心秘诀。
- **抗噪音（提升鲁棒性）：**
  - **cDPO / rDPO：** 引入标签平滑机制，容忍人工标注数据中的错误和主观偏见，防止模型过度拟合（钻牛角尖）。

第三章：GRPO

GRPO（Group Relative Policy Optimization，**组相对策略优化**）是 DeepSeek-R1 的核心，通过一个天才替换把 PPO 的四模型精简为三模型。

3.1 核心动机：干掉 Critic

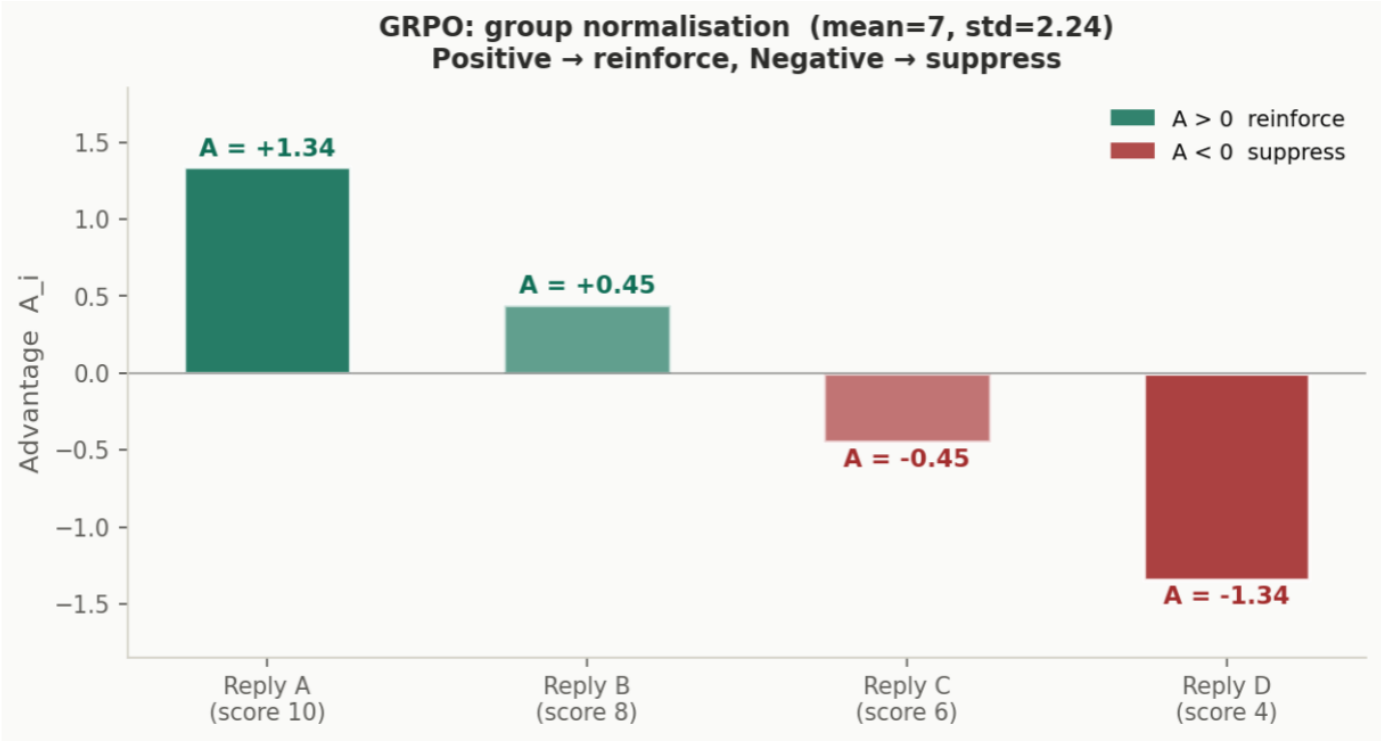
PPO 的 Critic 需要在每生成一个 token 时做一次前向传播预测 $V(s_t)$，开销极大。

GRPO 的核心替换：不和「历史预期」比（需要 Critic），只需和**「同班同学」**比——对同一道题让模型生成一组回答，组内相对排名。

3.2 组内标准化优势值

对同一 Prompt 采样 G 个回答，得分集合 $R = \{r_1, r_2, \ldots, r_G\}$：

$$A_i = \frac{r_i - \text{mean}(R)}{\text{std}(R)}$$



3.3 GRPO 目标函数

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G} \min(r_{t,i}(\theta)\cdot A_i,\ \text{clip}(r_{t,i}(\theta), 1-\varepsilon, 1+\varepsilon)\cdot A_i) - \beta \cdot D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\right]$$

其中 $r_{t,i}(\theta) = \dfrac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$ （token 级概率比值）。

> ✅ **和 PPO 唯一的本质区别（背这一句）**
>
> GRPO 和 PPO **唯一的本质区别**是 Advantage 的计算方式。PPO 靠吃显存的 Critic 预测绝对基准分，GRPO 靠组内均值标准化。Clip 截断和 KL 散度两大安全机制完全保留。

第四章：DAPO

DAPO（Decoupled Clip and Dynamic sAmpling Policy Optimization）针对 GRPO 在超长推理（Long-CoT）中的不足，共四项核心创新。

4.1 问题背景：策略熵坍塌

模型在长思考中找到一条关键推理捷径，该动作 $A \gg 0$，概率比值 $r_t = 1.8$。GRPO 的对称 Clip 冷酷地把 1.8 截断回 1.2，模型「不敢变聪明」，只输出安全但无信息量的「车轱辘话」。这叫**策略熵坍塌（Policy Entropy Collapse）**。

4.2 创新 ①：Clip-Higher（非对称截断）

$$\underbrace{\text{GRPO}}_{\text{对称}} : \text{clip}(r_t,\ 0.8,\ \mathbf{1.2}) \qquad \underbrace{\text{DAPO}}_{\text{非对称}} : \text{clip}(r_t,\ 0.8,\ \mathbf{1.28})$$

$\varepsilon_{\text{low}} = 0.2,\ \varepsilon_{\text{high}} = 0.28$ （论文实验值）

注意：这个非对称范围对所有 token **一视同仁**，无论 A 正负都用 $[0.8, 1.28]$，不是根据 A 的符号动态切换。

为什么提高上界能防止熵坍塌？ 对低概率但关键的推理路径（$\pi_{\text{old}} = 0.01$ 提升到 $\pi_\theta = 0.018$，则 $r_t = 1.8$），GRPO 只给 20% 强化空间，DAPO 给 28%。更高上界让模型敢于大胆强化低概率的关键探索 token。

4.3 创新 ②：Dynamic Sampling（动态采样）

过滤掉组内全对或全错的 Prompt：若全部得分相同，$\text{std}(R) = 0$，则 A_i 退化为 0，梯度全为 0，算力浪费。

> 🚨 **实验警告（面试加分项）**
>
> 受控实验发现**禁用 Dynamic Sampling 时 DAPO 表现反而更好**。原因：

1. 寻找「有意义 Prompt」需要 ~3× 额外采样，算力代价远大于收益
2. 对连续分数奖励几乎无用，全对/全错概率极低
3. 可被课程学习（调整难度分布）更高效地替代

4.4 创新 ③：Token-Level Loss（修复长度偏差）

GRPO 的问题（样本级平均）：

$$\mathcal{L}_{\text{GRPO}} = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\ell_{i,t}$$

若回答 A 有 200 token，回答 B 有 10 token：

回答	token 数	每个 token 的最终权重
A	200	$\frac{1}{2} \times \frac{1}{200} = \frac{1}{400}$

B	10	$\frac{1}{2} \times \frac{1}{10} = \frac{1}{20}$

回答 B 每个 token 的权重是 A 的 **20 倍**！后果：正确时短回答被大力强化（不鼓励长推理），错误时长回答惩罚极小（模型发现「答错了堆字数可以少扣分」）。

DAPO 的修复（全局 token 均等）：

$$\mathcal{L}_{\text{DAPO}} = \frac{1}{\sum_i |o_i|} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \ell_{i,t}$$

修复后所有 token 权重均为 $\frac{1}{200+10} = \frac{1}{210}$，完全平等。

4.5 创新 ④：Overlong Reward Shaping（超长惩罚）

若只设置硬截断无惩罚，模型会「无限堆废话反正不扣分」。DAPO 对超出 $L_{\max}$ 被截断的回答施加软性惩罚：

$$R_{\text{final}} = \begin{cases} R_{\text{RM}} & \text{若 } L \le L_{\max} \\ R_{\text{RM}} - \lambda \cdot \frac{L - L_{\max}}{L_{\max}} & \text{若被截断} \end{cases}$$

4.6 四大创新总览

创新点	解决的问题	实验结论
① Clip-Higher	策略熵坍塌，好思路被封顶	✅ 有效
② Dynamic Sampling	全对/全错零梯度浪费算力	⚠ 受控实验中禁用反而更好
③ Token-Level Loss	梯度偏向短序列，长推理学习慢	✅ 有效
④ Overlong Reward Shaping	超长截断无惩罚被钻空子	✅ 有效

第五章：Dr.GRPO

Dr.GRPO（**GRPO Done Right**）专门修补原版 GRPO 公式里两个隐藏的系统性偏差。

5.1 偏差一：长度偏差（Length Bias）

原版 GRPO 对每个 token 的损失除以序列长度 $|o_i|$：

$$\text{每 token 权重} = \frac{1}{|o_i|}$$

双向都出错：

情况	短回答 $L = 10$	长回答 $L = 1000$
正确（$A > 0$）	梯度大 → 被大力强化	梯度极小 → **正确长推理学习慢**
错误（$A < 0$）	惩罚大	惩罚≈0 → **模型堆字数逃避惩罚**

Dr.GRPO 修正： 改用固定常数 L（通常取 `max_completion_length`）：

$$\text{每 token 权重} = \frac{1}{L} \quad (L \text{ 全局固定不变})$$

5.2 偏差二：难度偏差（Difficulty Bias）

原版 GRPO 对 Advantage 除以标准差归一化：

$$\hat{A}_i = \frac{r_i - \text{mean}(R)}{\text{std}(R)}$$

对极难题（组内几乎全错，偶尔一个答对），$\text{std}(R) \to 0$，一除就变成 $\pm\infty$，梯度爆炸。即使不爆炸，也会给极难题赋予不成比例的超大梯度权重。

Dr.GRPO 修正： 去掉 std 归一化，只做均值中心化：

$$\hat{A}_i = r_i - \text{mean}(R)$$

> ⚠ **已知代价**：对「9 次错 1 次对」的极难题，那唯一一次正确的 Advantage 值相对较小（没被 std 放大），可能导致这类宝贵信号被低估。这是 Dr.GRPO 的 tradeoff，面试时主动说出来可以加分。

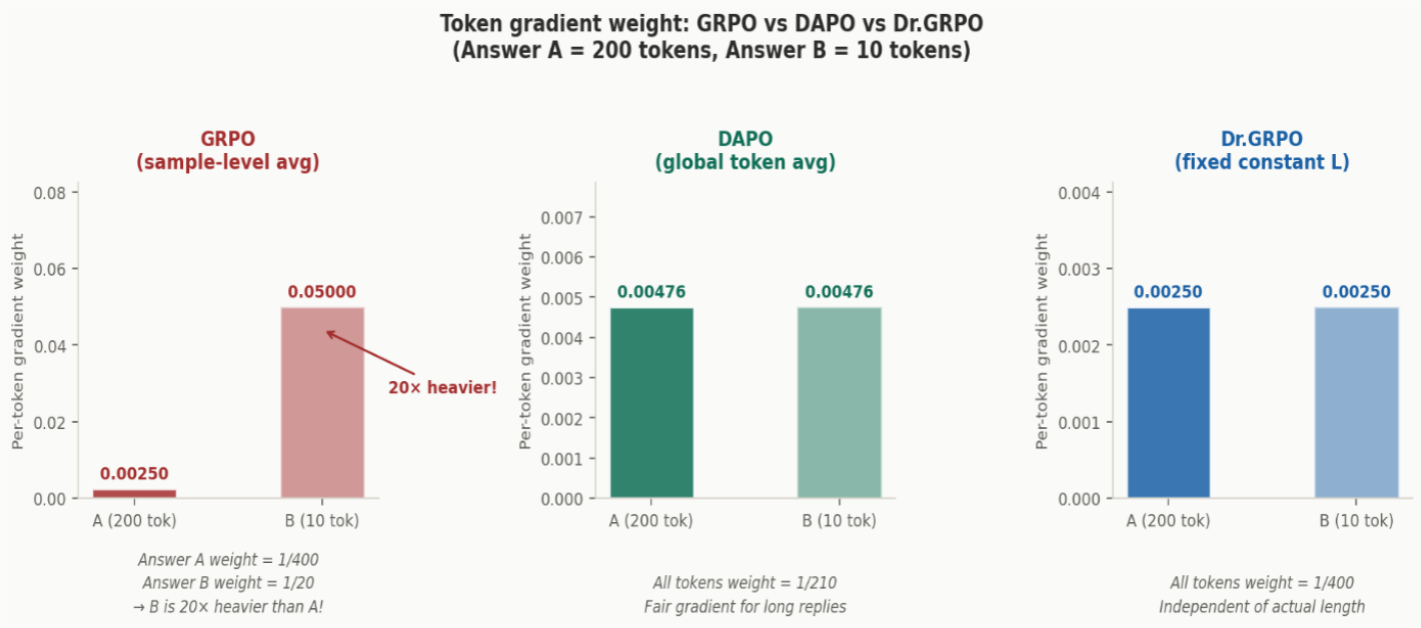
5.3 Dr.GRPO 完整目标函数

$$\mathcal{L}_{\text{Dr.GRPO}}(\theta) = -\frac{1}{L \cdot G} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \hat{\rho}_{i,t}(\theta) \cdot \big(r_i - \mathbb{E}[r_i]\big)$$

L = 固定的最大序列长度常数 | $\hat{\rho}_{i,t}$ = clip 后的重要性采样权重

实验效果： 错误回答的平均长度减少了 **38%**。

5.4 三种方法 Token 权重横向对比



方法	损失归一化	Advantage 是否除以 std	修复情况
GRPO	÷ $	o_i	$（各自序列长度）
DAPO	÷ $\sum	o_i	$（全局 token 总数）
Dr.GRPO	÷ L （固定常数）	❌ 否	**根治两个偏差**

第六章：GSPO

GSPO（Group Sequence Policy Optimization）是目前最适合训练超大规模 MoE 推理模型的 RL 算法，是 Qwen3 霸榜的核心底气。

6.1 GRPO 的两个致命缺陷

缺陷一：奖励粒度与优化粒度严重错位

- **优化是 Token 级的**：GRPO 逐个 token 算 r_t 、逐个 clip、逐个更新。
- **奖励是 Sequence 级的**：「代码跑通了没有」是整条回答的 0/1 信号。

拿「整句话的分数」强行分配给每个 token——相当于用「期末总评」评估「某次课举手发言的质量」。长推理时噪声不断累积，训练极易崩溃。

缺陷二：MoE 专家崩塌（Expert Collapse）

GRPO 的 token 级梯度极不稳定，容易把某几个专家的权重「打飞」。工程师被迫使用繁琐的 Routing Replay（路由回放）补丁：记录旧策略的路由记录并强制回放，代价是额外显存和代码复杂

度爆炸。

6.2 GSPO 的解法：Token 级 → Sequence 级

既然奖励是按整句话给的，重要性采样和截断也必须按整句话来算！

Step 1：计算整句联合概率比（除以 L 防数值爆炸）：

$$R_{\text{seq}} = \frac{1}{L}\sum_{t=1}^{L}\log\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)} = \frac{1}{L}\log\frac{\pi_\theta(\mathbf{y} \mid \mathbf{x})}{\pi_{\text{old}}(\mathbf{y} \mid \mathbf{x})}$$

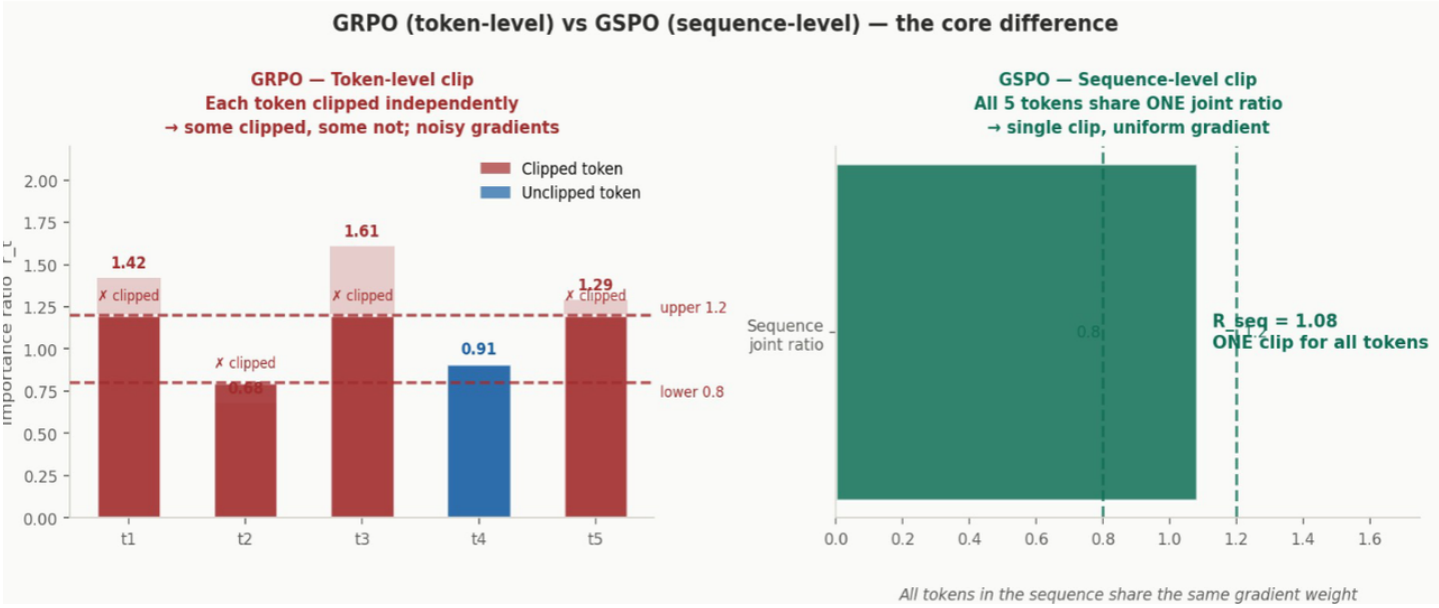
Step 2：对序列级比值做**一次** clip（整句一刀，不逐字截）：

$$R_{\text{clipped}} = \text{clip}(R_{\text{seq}},\ 1-\varepsilon,\ 1+\varepsilon)$$

Step 3：整句所有 token **共享同一权重**更新：

$$\mathcal{L}_{\text{GSPO}} = -A \cdot R_{\text{clipped}}$$

整条轨迹稳 → 所有字一起稳步更新；整条轨迹偏远 → 整个一起踩刹车。



6.3 三大收益

① 彻底拯救 MoE 训练

序列级优化抹平 token 级抖动噪声。Expert 们「同进同退」，不会被单个 token 的异常梯度打飞。工程上直接扔掉 Routing Replay 等复杂补丁。

② 完美的 Scaling Law

算力越多 → 性能持续上升；序列越长 → 推理质量持续提高；训练越久 → 无平台期和崩溃。

③ 训推分离架构友好（面试硬核加分项） ⭐⭐⭐

GRPO 需要把数据搬回训练引擎逐字重算 Logprobs（代价极高）。GSPO 只需推理引擎返回「整句话的粗粒度概率」（一个标量），省掉整个搬运和重算步骤，为训练集群和推理集群完全分离扫清了最大障碍。

6.4 GRPO vs GSPO 完整对比

维度	GRPO（Token 级）	GSPO（Sequence 级）
重要性采样粒度	逐 token 单独计算	整句联合概率比
奖励/优化匹配	❌ 错位（高方差）	✅ 完全匹配
MoE 训练	需 Routing Replay 补丁	直接稳定收敛
Scaling Law	步数长后容易 Collapse	算力越多性能越强
训推分离	低（需逐字重算 Logprobs）	高（粗粒度概率即可）
代表模型	DeepSeek-R1	Qwen3 系列

第七章：面试速查

算法全景对比

算法	模型数	Advantage 计算	约束机制	核心缺点
PPO	4	Critic 预测 $V(s)$	对称 Clip + 显式 KL	复杂易崩，显存极高
DPO	2	无（离线）	隐式数学 KL	过拟合；标注贵
GRPO	3	组内均值 + std 归一化	对称 Clip + 组内 KL	长推理探索受限
DAPO	3	同 GRPO（Token-Level 改良）	非对称 Clip（弱化 KL）	超参调整复杂
Dr.GRPO	3	去掉 std 归一化	固定常数归一化	极难题信号可能被低估
GSPO	3	组内均值（同 GRPO）	**Sequence 级 Clip**	实现复杂度较高
RLOO	3	留一法均值（不含自己）	对称 Clip + KL	效果与 GRPO 相近

RLOO 和 GRPO 的区别

$$A_i^{\text{RLOO}} = r_i - \frac{1}{G-1}\sum_{j\neq i} r_j$$

RLOO 用**留一法**（不含自己的均值），不除以标准差。GRPO 用全组均值（含自己），并除以标准差归一化。

RLVR — 可验证奖励

RLVR 是 DeepSeek-R1 的核心范式，**用程序可验证的规则代替 Reward Model**：

代码块

```
# 数学题
reward = 1.0 if extract_answer(output) == ground_truth else 0.0

# 代码题
reward = 1.0 if all(run_test(output, tc) for tc in test_cases) else 0.0

# 格式奖励（辅助信号）
reward += 0.1 if "<think>" in output and "</think>" in output else 0.0
```

维度	RLHF（RM 打分）	RLVR（可验证奖励）
奖励来源	神经网络近似	精确规则
Reward Hacking 风险	高（可找 RM 漏洞）	极低（规则精确）
标注成本	高（需人工偏好数据）	极低（自动验证）
适用场景	通用对齐、文科类	数学、代码、逻辑推理

> ✅ **DeepSeek 为什么成本极低？** GRPO（省 Critic）+ RLVR（省 RM 标注成本）组合拳。

一句话八股文速背

PPO：四模型（Actor/Critic/RM/Reference）；靠 Critic 估分，靠 Clip 截断，靠 KL 散度和 Reference 防走火入魔。痛点：Critic 极吃显存，四模型合计 150 GB+。

DPO：扔掉 RM 和 Critic，只用「好坏对」离线数据直接优化，两个模型就够。隐式 KL 内嵌在 β 里。但容易过拟合，不适合推理类模型。

GRPO：砍掉 Critic，靠对同一 Prompt 采样一组回答、用组内均值和标准差标准化得到 Advantage。Clip、KL 原样保留，适合数学/代码推理。

DAPO：非对称截断（上界 0.2→0.28）打破策略熵坍塌；Token-Level Loss 修复梯度偏向短序列的问题；Overlong Reward Shaping 防止模型堆字数钻空子。

Dr.GRPO：修正 GRPO 两个偏差：①长度偏差——除以 $|o_i|$ 改为除以固定常数 L；②难度偏差——去掉除以 $\mathrm{std}(R)$ 的归一化。实验效果：错误回答长度减少 38%。

GSPO：把 GRPO 的 token 级重要性采样升维为 sequence 级，从根本上解决粒度错位高方差问题。整条轨迹共享同一梯度权重，彻底拯救 MoE 专家崩塌，具备完美 Scaling Law，为训推分离扫清障碍。Qwen3 核心底气。

RLOO：Advantage = 当前得分 − 组内其他回答均值（不含自己）。与 GRPO 区别：留一法，不除以标准差。

高频面试 TOP 12

#	题目	难度
1	PPO 为什么需要 4 个模型，每个作用是什么？	⭐⭐
2	PPO-CLIP 为什么取 `min` 而不是直接截断？	⭐⭐⭐
3	GAE 是什么？$\lambda = 0$ 和 $\lambda = 1$ 分别退化为什么？	⭐⭐⭐
4	DPO 的隐式 KL 是什么意思？	⭐⭐⭐⭐
5	为什么 DeepSeek 训推理模型不用 DPO？	⭐⭐⭐
6	GRPO 如何不要 Critic 也能算 Advantage？	⭐⭐⭐
7	DAPO 的 Clip-Higher 怎么解决策略熵坍塌？	⭐⭐⭐⭐
8	Dr.GRPO 修了 GRPO 的哪两个 bug？	⭐⭐⭐⭐
9	GSPO 和 GRPO 最核心的区别是什么？	⭐⭐⭐⭐
10	什么是 Reward Hacking？如何防止？	⭐⭐
11	RLOO 和 GRPO 的区别？	⭐⭐⭐⭐

12	7B 模型全参数 PPO 大约需要多少显存?	⭐⭐⭐⭐

本手册综合自 2025 年大陆大模型算法岗面试真实考点